

MasterCube: A Family of Cross-Platform Hash Functions with AndRX Operations

Zheng Gong¹, Di Li¹,
Yufeng Tang², Dachao Wang³, Liangju Zhao^{1†}

¹ School of Computer Science, South China Normal University, China
cis.gong@gmail.com, edu.lidi97@gmail.com, liangjzhao@126.com

² Institute for Network Sciences and Cyberspace, Tsinghua University, China
yufengtang@tsinghua.edu.cn

³ Department of Electrical and Information Technology, Lund University, Sweden
mr.ongor@gmail.com

Abstract. MasterCube is a family of cryptographic hash functions designed to be efficient and secure across various platforms. The design of MasterCube is based on the sponge construction, allowing for variable output lengths and potential extensions for authenticated encryption, key derivation and pseudorandom bit generation. Instead of the commonly used permutation-based function for the iteration, MasterCube uses a transformation Cube-f which is inspired by the ZIP construction. Specifically, Cube-f utilizes a combination of permutation-based function Cube-p, which performs a mix of AND, Rotation, and XOR (AndRX) operations on its 32-bit lanes. The combination of those design choices results in strong security and enjoys the compatibility of different length of registers in cross-platform CPUs, and the tensor-based white-box implementation in keyed scenarios. In this document, we present the specification of MasterCube, analyze its security properties, and provide draft implementation results on different platforms. We also provide a preliminary comparison of MasterCube with standardized hash functions in terms of performance and security.

Keywords: Hash function, Sponge construction, AndRX, Cross-platform, Analysis

[†]The authors are listed in alphabetic order. Document Version 2.0, June 2026.

Contents

1	Definitions and symbols	3
1.1	Definitions	3
1.2	Symbols and variables	3
2	Specification of MasterCube	4
2.1	The Cube-p permutation	5
2.2	The Cube-f transformation	8
2.3	The family of MasterCube hash functions	9
3	The design rationale	10
3.1	Using transformation in sponge	10
3.2	Transformation from ZIP construction	10
3.3	Involutory operations	11
3.4	Arrangement of the state	11
3.5	Parameters for cyclic shifts and MAndRX	11
3.6	Cross-platform design	12
4	Security analysis	12
4.1	Security of Cube-p	13
4.2	Security of Cube-f	14
4.3	Security of MasterCube	14
4.4	Indifferentiability	16
4.5	Length extension attack	16
4.6	Quantum security analysis	16
4.7	Statistical randomness tests	18
5	Implementations	19
5.1	Software implementations	19
5.2	Hardware implementations	22
5.3	Tensor-based implementation of keyed Cube-f	24
A	Test Vectors for MasterCube	27
B	The tensor-based implementation of keyed Cube-f	27
B.1	White-box cryptography	27
B.2	Algebraic formulation of encoded lane operations	28
B.3	Security analysis against structural ASA decomposition	30
B.4	Performance evaluation of the tensor-based implementation	31

1 Definitions and symbols

To ensure a precise and unambiguous description, this section defines the terminology and mathematical symbols used in **MasterCube**.

1.1 Definitions

Bit string A sequence of binary digits (0 or 1). The leftmost bit is the first bit.

Sponge construction A framework that builds a function mapping variable-length inputs to variable-length outputs utilizing a fixed-length state permutation or transformation.

ZIP construction A framework that builds a pseudorandom function by using a known permutation.

Rate (r) The number of bits of the state array that interact with the input message block in each iteration of the Sponge construction.

Capacity (c) The portion of the internal state that is not directly altered by the input message during the absorbing phase, determining the security level of the hash function.

Block A bit string whose length is the rate (r) of the sponge construction. Messages are divided into blocks before being absorbed.

Cell A 16-bit string. In **MasterCube**, a 32-bit Lane is horizontally partitioned into two Cells, forming the branches for the Feistel-like AndRX network.

Lane A 32-bit string corresponding to a fixed (x, y) coordinate in the state array.

Padding A rule for appending additional bits to a message so that its length becomes an exact multiple of the block size (r).

Digest / Hash Value The final output bit string truncated to the desired target length.

Cube-p The permutation that is used in the design and analysis of **MasterCube**.

Cube-f The transformation that is used in **MasterCube**.

1.2 Symbols and variables

The following mathematical symbols and string operations are utilized to describe the **MasterCube** algorithm.

- 0^k : A bit string consisting of k consecutive zeros.
- $|M|$: The number of blocks of the bit string M .
- $X \parallel Y$: The concatenation of bit string X followed by bit string Y .
- $X \oplus Y$: The bitwise exclusive-OR (XOR) addition of two bit strings X and Y of equal length.
- $X \wedge Y$: The bitwise logical AND operation of two bit strings X and Y .
- $\neg X$: The bitwise complement (NOT) of bit string X .
- $X \lll k$: The logical cyclic shift (rotation) of the string X to the left by k bit positions.
- $X \ggg k$: The logical cyclic shift (rotation) of the string X to the right by k bit positions.

2 Specification of MasterCube

MasterCube is a family of cryptographic hash functions based on the sponge construction, which supports variable output lengths. The input message is processed in blocks, and the sponge construction is used to produce the final hash value. Different from common permutation-based ciphers, a transformation called **Cube-f** is used in MasterCube to improve the security level. The **Cube-f** function is constructed from a permutation called **Cube-p**, which is inspired by the ZIP construction [13]. The overview of MasterCube is depicted in Figure 1. In this specification, we propose three instances of MasterCube, and the parameters of sponge are given in Table 1. In this section, we will describe the design of **Cube-p** first, then show how to construct **Cube-f**, and finally provide the parameters used in MasterCube.

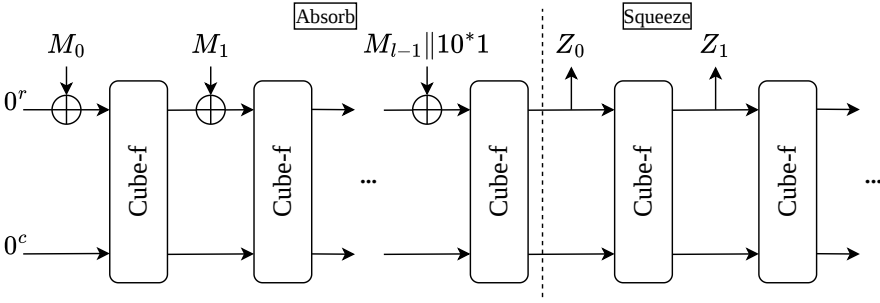


Figure 1: Overview of MasterCube.

Table 1: Rates and capacities (bits) for the proposed MasterCube instances.

	MasterCube-512	MasterCube-768	MasterCube-1024
rate (block size)	960	704	448
capacity	576	832	1088
hash size	512	768	1024

The state. The underlying transformation in MasterCube takes a round function called **Cube-f** as a building block. The state of **Cube-f** is denoted by A , which consists of 6×8 lanes. Each lane is a 32-bit vector. It can be cubically depicted as Figure 2.

A lane is denoted by $A[x, y]$, $x \in \{0, 1, \dots, 7\}$ and $y \in \{0, 1, \dots, 5\}$, where $x = 0$ is at the left and $y = 0$ is at the bottom. Each lane is split into two 16-bit cells, which forms the z -axis. Therefore, each cell is denoted by $A[x, y, z]$, $x \in \{0, 1, \dots, 7\}$, $y \in \{0, 1, \dots, 5\}$, and $z \in \{0, 1\}$.

A slice is denoted by A_{xy}^i , $i \in \{0, 1\}$, which contains all cells $A[x, y, i]$ where $x \in \{0, 1, \dots, 7\}$ and $y \in \{0, 1, \dots, 5\}$. The whole state A can be viewed as two slices on the z -axis, A_{xy}^0 and A_{xy}^1 . Within each slice, each row has totally $16 \times 8 = 128$ bits. When it is implemented with SSE or NEON instruction set, such a slice is stored by six 128-bit registers. When AVX2/AVX512 is available, two consecutive rows can be stored in a 256-bit register. Therefore, a slice will occupy three 256-bit registers in AVX2/AVX512 implementations.

A sheet is denoted by A_{yz}^i , $i \in \{0, 1, \dots, 7\}$, which contains all cells $A[i, y, z]$ where $y \in \{0, 1, \dots, 5\}$ and $z \in \{0, 1\}$. The whole state can be viewed as 8 slices on the x -axis.

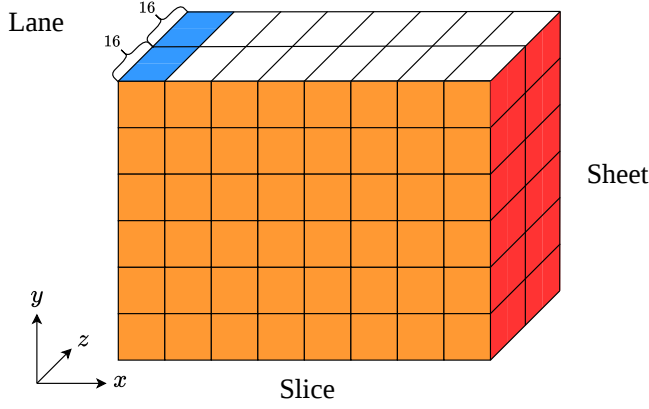


Figure 2: A cubical view of the Cube-f state.

Padding rule. We adopt the same padding rule as Keccak [14], shown in the following Definition 1, called *multi-rate padding*. In the rest of this specification, we assume that all messages to be absorbed have been padded with this rule, so the padding part will be ignored.

Definition 1. Multi-rate padding, denoted by pad_{10*1} , appends a single bit 1 followed by the minimum number of bits 0 followed by a single bit 1 such that the length of the result is a multiple of the block length.

2.1 The Cube-p permutation

The permutation Cube-p contains several operations in an iteration, which are listed below.

2.1.1 Non-linear operation on lanes: MAndRX

Each lane $A[x, y]$ has 32 bits and is split into two 16-bit halves, $A[x, y, 0]$ and $A[x, y, 1]$. A 6-round AndRX operation, called MAndRX, is then applied to achieve a full diffusion over these 32 bits. During the MAndRX operation, $A[x, y, 0]$ is viewed as the left half and $A[x, y, 1]$ is the right half.

One-round AndRX operation is depicted in Figure 3, which has three rotations with shifts (α, β, γ) . Mathematically, given a 32-bit value $X = L \parallel R$ where L and R are 16-bit values, one-round AndRX operation can be described as follows.

$$\begin{cases} L' = (L \ggg \alpha) \wedge (L \ggg \beta) \oplus (L \ggg \gamma) \oplus R \\ R' = L \end{cases}$$

In MAndRX, each round of AndRX has its independent shifts for better diffusion properties. We choose $\{(0, 1, 8), (14, 5, 0), (9, 12, 8), (14, 5, 0), (0, 1, 8), (8, 9, 0)\}$ corresponding to each round of AndRX operations in MAndRX.

2.1.2 Linear operation on slices: ShiftRows and MixRows

There are two linear operations on slice, shown in Figure 4. The first operation is ShiftRows. In Cube-p, the slice A_{xy}^0 shifts to the left while A_{xy}^1 shifts to the right. This aims to break the connection between the two halves of a lane in the AndRX operations. Shifting to

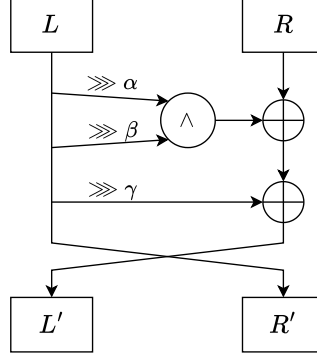


Figure 3: A Feistel structure of AndRX for MAndRX.

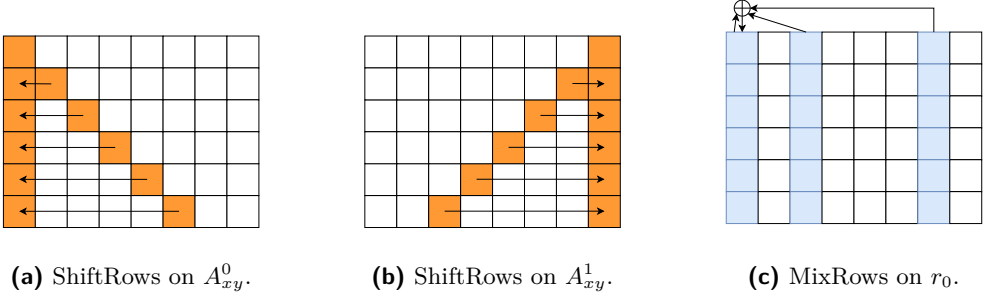


Figure 4: Linear operation on slices: ShiftRows and MixRows.

the same direction would preserve the lanes. The operation is shown in Figure 4a and Figure 4b.

The second operation is MixRows, which mixes cells within each row using a 3-XOR operand. Denote a row in Figure 4c as a vector $[r_0 \ r_1 \ r_2 \ r_3 \ r_4 \ r_5 \ r_6 \ r_7]^T$, where each r_i is 16-bit. Consequently, MixRows can be represented as the following multiplication by a cyclic matrix.

$$M_{MixRows} = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}, \quad \begin{cases} [r'_0 & r'_2 & r'_4 & r'_6]^T = M_{MixRows} \cdot [r_0 & r_2 & r_4 & r_6]^T \\ [r'_1 & r'_3 & r'_5 & r'_7]^T = M_{MixRows} \cdot [r_1 & r_3 & r_5 & r_7]^T \end{cases}$$

2.1.3 Linear operations on sheets: SwapRows, MixColumns, and CrossMixs

We note that the non-linear operation on lanes and the linear operations on slices do not mix between different rows. On sheets, three linear operations are applied to further break the potential invariant or symmetry properties between rows. These operations are shown in Figure 5. For simplicity, we denote the cell in a sheet by v_i where $i \in \{0, 1, \dots, 5\}$ and v_i is 16-bit.

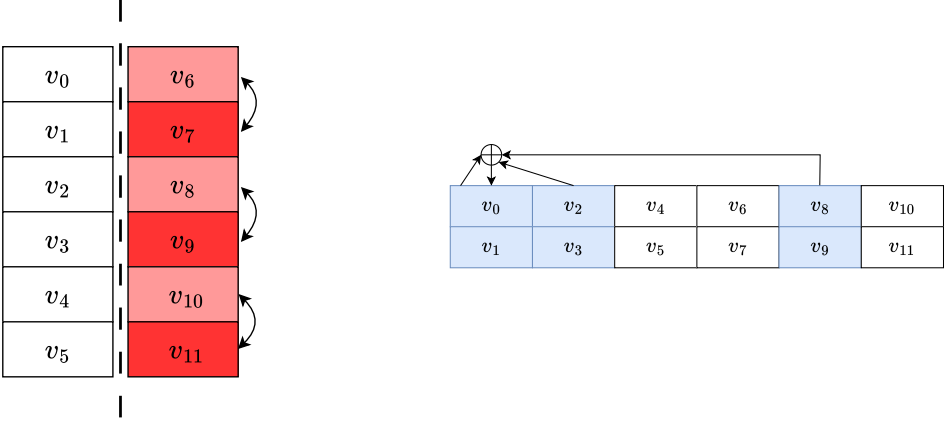
The first operation is SwapRows, which swaps rows v_6 and v_7 , v_8 and v_9 , as well as v_{10} and v_{11} in the second column, shown in Figure 5a.

The second operation is CrossMixs. It rearranges the sheet as two vectors $[v_0 \ v_2 \ v_4 \ v_6 \ v_8 \ v_{10}]^T$ and $[v_1 \ v_3 \ v_5 \ v_7 \ v_9 \ v_{11}]^T$, and then applies the same mixing operation to each of them.

Mathematically, the mixing operation is a multiplication by the following matrix, which is cyclic and involutory.

$$M_{CrossMixs} = \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 \end{bmatrix}$$

$$\begin{cases} [v'_0 & v'_2 & v'_4 & v'_6 & v'_8 & v'_{10}]^T = M_{CrossMixs} \cdot [v_0 & v_2 & v_4 & v_6 & v_8 & v_{10}]^T \\ [v'_1 & v'_3 & v'_5 & v'_7 & v'_9 & v'_{11}]^T = M_{CrossMixs} \cdot [v_1 & v_3 & v_5 & v_7 & v_9 & v_{11}]^T \end{cases}$$



(a) The SwapRows.

(b) The CrossMixs.

Figure 5: Linear operations on sheets: SwapRows and CrossMixs.

The third operation is MixColumns, which operates by matrix multiplications. Because the matrices are not cyclic and thus not easy to depict as figures. We then describe it formally as matrix multiplications in below.

$$M_{MixColumns}^1 = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad \begin{cases} [v'_0 & v'_2 & v'_4]^T = M_{MixColumns}^1 \cdot [v_0 & v_2 & v_4]^T \\ [v'_1 & v'_3 & v'_5]^T = M_{MixColumns}^1 \cdot [v_1 & v_3 & v_5]^T \end{cases}$$

$$M_{MixColumns}^2 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad \begin{cases} [v'_6 & v'_8 & v'_{10}]^T = M_{MixColumns}^2 \cdot [v_6 & v_8 & v_{10}]^T \\ [v'_7 & v'_9 & v'_{11}]^T = M_{MixColumns}^2 \cdot [v_7 & v_9 & v_{11}]^T \end{cases}$$

2.1.4 The round constants

To avoid the slide attacks and break the rotation invariant within each lane, **Cube-p** uses the i th-round constant $RC[i] = [0x243F \oplus i, 0x6A88 \oplus i, 0x85A3 \oplus i, 0x08D3 \oplus i, 0x1319 \oplus i, 0x8A2E \oplus i, 0x0370 \oplus i, 0x7344 \oplus i]$. The constants in this vector are generated from the hexadecimal representation of the fraction part of the constant $\pi = 3.1415926 \dots$. In round i , **Cube-p** XORs $RC[i]$ to the first row of A_{xy}^0 .

2.1.5 The round functions and Cube-p algorithm

In the design of MasterCube, we derive security properties from a permutation called Cube-p. We emphasize that Cube-p is used for the ZIP construction of Cube-f and its security analysis. Cube-p is an iterative permutation of which the round function is built with aforementioned operations. The pseudocode of the round function and its inverse are shown in Algorithm 1 and Algorithm 2, respectively. The algorithm Cube-p is illustrated in Algorithm 3.

Algorithm 1 The Cube-p round function.

```

1: procedure CUBE-P-R( $A, RC$ )
2:    $A \leftarrow \text{CrossMixs}(A)$ 
3:    $A \leftarrow \text{SwapRows}(A)$ 
4:    $A \leftarrow \text{MAndRX}(A)$ 
5:    $A \leftarrow \text{CrossMixs}(A)$ 
6:    $A \leftarrow \text{SwapRows}(A)$ 
7:    $A \leftarrow \text{MixColumns}(A)$ 
8:    $A \leftarrow \text{ShiftRows}(A)$ 
9:    $A \leftarrow \text{MixRows}(A)$ 
10:   $A \leftarrow A \oplus RC$ 
11:  return  $A$ 
12: end procedure

```

Algorithm 2 The inverse of Cube-p round function.

```

1: procedure CUBE-P-IR( $A, RC$ )
2:    $A \leftarrow A \oplus RC$ 
3:    $A \leftarrow \text{MixRows}(A)$ 
4:    $A_{xy}^0, A_{xy}^1 \leftarrow A_{xy}^1, A_{xy}^0$ 
5:    $A \leftarrow \text{ShiftRows}(A)$ 
6:    $A \leftarrow \text{MixColumns}(A)$ 
7:    $A_{xy}^0, A_{xy}^1 \leftarrow A_{xy}^1, A_{xy}^0$ 
8:    $A \leftarrow \text{SwapRows}(A)$ 
9:    $A \leftarrow \text{CrossMixs}(A)$ 
10:   $A_{xy}^0, A_{xy}^1 \leftarrow A_{xy}^1, A_{xy}^0$ 
11:   $A \leftarrow \text{MAndRX}(A)$ 
12:   $A \leftarrow \text{SwapRows}(A)$ 
13:   $A \leftarrow \text{CrossMixs}(A)$ 
14:  return  $A$ 
15: end procedure

```

2.2 The Cube-f transformation

Given Cube-p in the previous subsection, we turn it into a transformation by using the ZIP construction in [13]. In the ZIP construction, one branch takes the round function of the original permutation while the other branch runs the inverse of the round function, shown in Figure 6. Since most operations are involutory, the inverse of Cube-p is easy to construct. Only two operations need special attention, which are MAndRX and ShiftRows.

1. The MAndRX operation is a small Feistel cipher, its inverse can be derived by swapping the two halves and feed into the rounds in the reversed order.

Algorithm 3 The Cube-p permutation.

```

1: procedure CUBE-P( $A$ )
2:   for  $i \leftarrow 0$  to 17 do
3:      $A \leftarrow \text{Cube-p-r}(A, RC[i])$ 
4:   end for
5:   return  $A$ 
6: end procedure

```

2. The ShiftRows operation itself is not involutory, but the operations on A_{xy}^0 and A_{xy}^1 are the inverse of each other. Therefore, to get the reversed ShiftRows, we can swap two slices, apply ShiftRows, and swap slices again.

The pseudocode for Cube-f is given in Algorithm 4.

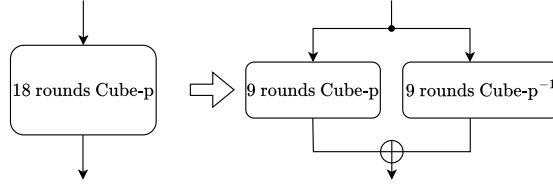


Figure 6: The ZIP construction of Cube-f.

Algorithm 4 The Cube-f transformation.

```

1: procedure CUBE-F( $A$ )
2:    $A' \leftarrow A$ 
3:   for  $i \leftarrow 0$  to 8 do
4:      $A \leftarrow \text{Cube-p-r}(A, RC[i])$ 
5:   end for
6:   for  $i \leftarrow 0$  to 8 do
7:      $A' \leftarrow \text{Cube-p-ir}(A', RC[i])$ 
8:   end for
9:    $A \leftarrow A \oplus A'$ 
10:  return  $A$ 
11: end procedure

```

2.3 The family of MasterCube hash functions

MasterCube is constructed by organizing components from the previous subsections in the sponge construction, shown in Figure 1. In this specification, we propose MasterCube-512/768/1024. The state has 1536 bits, and the claimed security levels for each instance are given in Table 2. Due to the sponge construction and the padding rule, the input message can support $2^{64} - 1$ bits at most.

Table 2: Claimed security levels for the proposed MasterCube instances.

	MasterCube-512	MasterCube-768	MasterCube-1024
(second) preimage	512	768	1024
collision	256	384	512

3 The design rationale

3.1 Using transformation in sponge

While most sponge-based hash functions are built based on a permutation, the sponge construction itself is defined for both permutations and transformations [5]. Note that the key difference between an n -bit permutation and an n -bit transformation is that the transformation is non-invertible. Using a transformation in the sponge construction provides a better security level against general preimage and second preimage attacks. The workload of both attacks are 2^{c-1} and $\frac{2^c}{|M|}$, where c is the capacity and $|M|$ is the number of message blocks. As a comparison, the workloads of the same attacks are both $2^{\frac{c}{2}}$ in a permutation-based sponge construction. Thus, transformation-based sponge constructions have a higher security level with the same size of capacity. In particular, to achieve 1024-bit security for a 1024-bit hash function, MasterCube uses a 1536-bit state while a permutation-based design would require at least 2048 bits.

On the other hand, a permutation-based sponge construction is more secure against general attacks leveraging cycles in the output. This feature is important when we need long outputs. For example, when the output is used as a keystream for encryption, cycles can be turned into distinguishers. However, in applications of hash functions, the required outputs are of a small fixed size. Particularly, our scenario considers at most 1024-bit outputs. As a result, it is infeasible to find cycles within such short outputs.

Due to these security reasons, we chose to use a transformation as the internal function. To determine the required capacity size for each MasterCube instance, we first consider the largest possible $|M|$. Our scenario has messages of length $2^{64} - 1$ bits in maximum. Thus, given the rate as r , we have $|M| = \frac{2^{64}-1}{r} < 2^{64}$. Then, for an n -bit output hash, the n -bit security level against preimage and second preimage attacks implies the following derivation.

$$\begin{cases} 2^{c-1} \geq 2^n \\ \frac{2^c}{|M|} \geq 2^n \end{cases} \implies \begin{cases} c \geq n + 1 \\ c \geq n + \log_2(|M|) \end{cases}$$

For simplicity, we select $c = n + 64$ for all instances of MasterCube and derive the rate as $r = 1536 - c$, which are the values listed in Table 1.

3.2 Transformation from ZIP construction

Designing a secure transformation (i.e., a pseudorandom function) directly is generally more difficult than designing a permutation. In symmetric cryptography, the design and analysis of permutations are considerably more mature. A practical pseudorandom function (PRF) construction is that one first builds a strong permutation and then applies a proved PRP-PRF switch to obtain a secure transformation. Examples of such a switch could be Davie-Meyers construction and Sum-of-Permutation. This approach is theoretically attractive because it combines the better-understood cryptanalysis of permutations and the provable security of the switch.

In MasterCube, we follow the same philosophy as Keccak, namely the *hermetic sponge strategy*. Under this strategy, one does not require a proof of security for the PRP-PRF switch. Instead, the switch is required to exhibit no exploitable structural properties.

Consequently, we can primarily rely on cryptanalysis results and thus have more potential choices of the switch.

From this perspective, the recent ZIP construction proposed by [13] becomes our candidate, depicted in Figure 6. Given an n -round permutation P , ZIP construction evaluates two branches on the same input, which are the reduced $n/2$ -round permutation of P and its inverse. The outputs of these two permutations are then XORed to produce the final output of the ZIP construction. It is shown in [13] that differential, linear, and Meet-in-the-Middle attacks on the ZIP construction are equivalent to the same attacks on the underlying n -round permutation. This desirable property aligns well with the hermetic sponge strategy, and we can then focus on designing a secure permutation **Cube-p**. Compared with the Davie-Meyer construction, the ZIP construction also doubles the footprint of datapath but reduces the latency from inputs to outputs thanks to its parallel structure. Thus, this yields a time-memory trade-off that is more attractive for cryptographic engineering.

3.3 Involutory operations

In the ZIP construction, one branch uses the round function of the original permutation and the other one runs its inverse. This requires the inverse of the round function to be not too complicated to avoid extra implementation costs. We then decide to have involutory operations or something similar in **MasterCube**. As shown in Section 2, most operations are involutory and have lower implementation costs for both the round function and its inverse, except the MAndRX and ShiftRows.

The MAndRX on each lane is a 6-round Feistel cipher. It is well-known that the inverse of such an even-round Feistel cipher can be constructed by reversing the order of the round functions. The ShiftRows on slices A_{xy}^0 and A_{xy}^1 only differs in directions. Thus, its inverse can be implemented by swapping the two slices of both input halves, and then the ShiftRows operation is reused in both branches of the ZIP construction. As a result, the extra costs in the inverse branch is just a few swaps while they can be optimized in hardware and software platforms.

3.4 Arrangement of the state

The state of **Cube-p** is arranged in a way friendly to SIMD registers. Each slice can be viewed as 6×128 bits, which fits into 6 128-bit registers. In this way, the left halves and right halves of MAndRX are always in different registers, so the non-linear operation on 8 lanes in the same row can be run in parallel.

When 256-bit registers are available, two consecutive rows are put in the same 256-bit register. In this case, when an XOR operation happens between two 256-bit registers, there are two XOR operations being executed in two pairs of 128-bit values simultaneously. This becomes a minor restriction to our linear operations. Therefore, in CrossMix and MixColumns, the state is split into two independent subsets each of which takes the same operations. In order to break this independency, the SwapRow is added to exchange the entropy between these subsets. When it comes to 512-bit registers, the states of both branches of ZIP construction can be put on the same 512-bit registers. Thanks to involutory operations, the update of both branches is almost the same, and thus operations can be applied directly to the whole 512-bit registers, achieving fully parallel computation of branches.

3.5 Parameters for cyclic shifts and MAndRX

There are many cyclic shifts in **MasterCube**. The main part comes from MAndRX, and the others are from MixRows and ShiftRows. However, in software implementation, it

should be noted that there are no bit-oriented cyclic shifts in common SIMD instruction set (SSE/AVX on X86-64 and NEON on ARM). The only way to implement them is either using two normal shifts instruction followed by an OR instruction or relying on shuffle instructions when the amount of shifts is a multiple of 8. The former usually takes 2 CPU cycles, and the latter consumes 1 or 2 cycles depending on the shuffle. Thus, it is clear to see that reducing the number of cyclic shifts and using cyclic shifts by a multiple of 8 are better for software performance.

In MAndRX, we try to increase the number of 0 and 8 in α , β , and γ while the security of MAndRX should not be sacrificed too much. This is difficult and less effective, since a previous work has investigated all possible parameters for SIMON [16]. To compensate security when having restricted parameters, we use different cyclic shifts in different rounds in MAndRX. This idea provides a large parameter space which was not explored. During the search, we focus on the largest probability of differential trails and algebraic degree of MAndRX. The first step maximizes security based on both metrics. At the same time, we only keep candidates with a full diffusion. Among the candidates, the second step maximizes the number of 0 and 8, which also maximizes the performance. Finally, there are still multiple choices, and we take a random set of parameters for our MAndRX.

In MixRows and ShiftRows, we first restrict the amounts of shifts to multiples of 8. Shuffle instructions are then used to implement these operations, but there are still some particular tricks for further improvements. A shuffle instruction on bytes usually requires an extra register to store the shuffle, which may require memory loading. However, a shuffle instruction on 32-bit word can encode the shuffle directly inside itself, leading to zero cost for accessing memory. This can further reduce the actual cost of shuffle instructions in implementation. For example, ShiftRows on some rows shifts by 4 bytes and can be replaced by a ‘faster’ shuffle. In the design of MixRows, we also use only cyclic shifts by 32 bits to build the operation, so that it can be implemented efficiently.

3.6 Cross-platform design

MasterCube is designed to be efficient on a wide range of platforms, including both software and hardware. In software-oriented implementations, it is optimized for CPUs with SIMD instructions, such as Intel/AMD with AVX instructions and ARM processors with NEON instructions. The slice of **MasterCube** is deliberately designed to ensure that it is compatible with registers of different sizes from 32-bit to 512-bit. In hardware-oriented implementations, the non-linear operation AndRX is a lightweight component. Moreover, the ZIP construction of **Cube-f** allows for a parallel structure that can be efficiently implemented in both software and hardware. The choice of involutory operations and the arrangement of the state also contribute to the efficiency of the design across different platforms. Overall, the design of **MasterCube** is guided by the goal of achieving high security while maintaining efficiency across various platforms, making it a versatile choice for cryptographic applications.

4 Security analysis

To evaluate the security of **MasterCube**, we begin by analyzing **Cube-p**. The security of **Cube-f** is then estimated based on the combination of the ZIP construction and **Cube-p**, as detailed in the following subsections. Finally, the security of **MasterCube** comes from the combination of the sponge construction and **Cube-f**. Meanwhile, we also provide quantum security analysis. The classical cryptanalysis in this section is based on a Constraint-Programming (CP) solver called OR-Tools [20].

4.1 Security of Cube-p

4.1.1 Differential and linear cryptanalysis

The first set of analysis is the common differential and linear cryptanalysis. This aims to find the best differential and linear distinguishers. However, due to the large state of **Cube-p**, it is infeasible to search the best trails directly by feeding all details as a model to CP solvers. We then turn to simpler techniques to estimate the security level.

Cube-p is designed with the bit-slicing technique, and the MAndRX operation is designed to achieve a full diffusion over each 32-bit lane. This allows us to view it as a block cipher and count the number of active ‘Sboxes’, where each 6-round AndRX sub-cipher in MAndRX is regarded as an Sbox. Thus, in the CP model, each AndRX sub-cipher is modelled as

1. If the input is zero, the output is also zero;
2. If the input is non-zero, the output can take any non-zero value.

Meanwhile, thanks to the bit-slicing design, the whole linear layer can be simplified by consider only one bit, and we can find its branch number before entering further analysis. As a result, the whole linear layer in **Cube-p** gives branch number 11. The estimated numbers of active Sboxes are 11, 18, and 28 for 2, 3, and 4 rounds, respectively. This result is the same for both differential and linear cryptanalysis. The automatic search program spent 3 days on 4 rounds by using an AMD CPU with 48 threads. This is the longest trail that we could find within a reasonable time.

To estimate the security level of **Cube-p**, we assume that, in the worst case, attackers can concatenate best trails for short rounds to form optimal trails for longer rounds. Then, a 16-round **Cube-p** is split into 4 parts each of which has 4 rounds. By using the number of active Sboxes for 4 rounds, the best 16-round trail can have at most $4 \times 28 = 112$ active Sboxes. Meanwhile, the best differential trail of the 6-round AndRX sub-cipher has probability 2^{-10} , and the squared correlation of the best linear trail is also 2^{-10} . A 16-round **Cube-p** should have at least 1120-bit security against these attacks. Our estimated results are summarised in Table 3. For 18 rounds, the best trails can be estimated as the concatenation of 4-round trails and 2-round trail which gives $4 \times 28 + 11 = 123$ active Sboxes leading to complexity at least 2^{1230} . We emphasize that these security levels are conservative, the rounds of **Cube-p** might be reduced according to the further security investigation. Indeed, the result for 4 rounds exceeds twice that for 2 rounds ($28 > 2 \times 11$). This indicates that the concatenation assumption is unlikely to hold in full-round **Cube-p**, and consequently that the reported security levels underestimates the actual security.

Table 3: Estimated security level of **Cube-p** against differential/linear attacks.

number of rounds	2	3	4
number of active Sboxes	11	18	28
complexity of attacks	2^{110}	2^{180}	2^{280}

4.1.2 Algebraic and integral attacks

As a component in hash functions, **Cube-p** should provide a sufficiently high security level against algebraic and integral attacks. Common integral analysis requires constructing a mathematical model of the cipher and counting monomial trails in the Monomial Prediction technique. Luckily, the design of **Cube-p** provides a simple and effective method to estimate the security.

The 6-round sub-cipher in MAndRX is a 32-bit AndRX cipher whose boolean expressions of output bits can be computed. By modelling each input bit as a variable, we managed to derive the expressions by propagating variables through 6 rounds. Then, we found that the left half of the output has degree 29 and the right half has 20. Furthermore, each expression involves all 32 input variables, and each variable is involved in at least a term of degree 19. These observations imply that

1. after 1 round of **Cube-p**, each output bit has at least degree 20;
2. the 6-round AndRX sub-cipher achieves a full diffusion over the lane;
3. attacks involving any variables have to handle at least a term of degree 19.

These properties of MAndRX then lead to a quick increasing of algebraic degree in **Cube-p**. As a comparison, Keccak and Ascon only have a non-linear operation of degree 2, which slows down the growth of their degrees.

Based on our observations, after two round of **Cube-p**, the degree of each output bit will be at least $19 \times 20 = 380$, shown in Table 4. Furthermore, a 3-round **Cube-p** has degree closed to 1536, since $19 \times 20 \times 20 > 1536$. Integral attacks on **Cube-p** will then have a data complexity closed to 2^{1536} . Algebraic attacks will also handle a algebraic system with a large number of variables and high degrees for polynomials.

Table 4: Estimated algebraic degree of **Cube-p**.

number of rounds	1	2	3
algebraic degree	20	380	~ 1536

4.2 Security of **Cube-f**

When it comes to **Cube-f**, security against differential and linear attacks follows directly from the structure of the ZIP construction and the security of the original permutation [13]. Suppose attackers find a differential trail covering **Cube-f**. Then the trail must have the form shown in Figure 7a, where the input differences to both branches are identical, and the XOR of the two output differences equals the output difference of **Cube-f**. Note that, on the right branch, the differential trail from δ_i to $\alpha \oplus \delta_i$ implies a trail from $\alpha \oplus \delta_i$ to δ_i in the reverse direction. Therefore, such a trail on **Cube-f** is equivalent to a trail on **Cube-p**, as illustrated in Figure 7a. Hence, the security of **Cube-f** against differential attacks should be no weaker than that of **Cube-p**. A similar argument applies to linear trails, as shown in Figure 7b.

Integral and algebraic attacks now target half-round versions of **Cube-p**. However, as shown in the previous section, the rapid growth of the algebraic degree provides strong resistance against both attacks. Therefore, 9 rounds in each branch of **Cube-f** should be sufficient to prevent them.

4.3 Security of MasterCube

For MasterCube, the security against general preimage, second preimage, and collision attacks directly come from the combination of the ZIP construction and the sponge construction. The ZIP construction itself is a secure transformation. We then consider the security of a sponge construction with a transformation.

The notations used below are r , c , and n corresponding to rate, capacity, and digest length in bits, respectively. The construction is compared with a random oracle where its output is truncated to n bits. The state of the sponge has $c + r$ bits, where we also call the rate part the inner state and the capacity part the outer state.

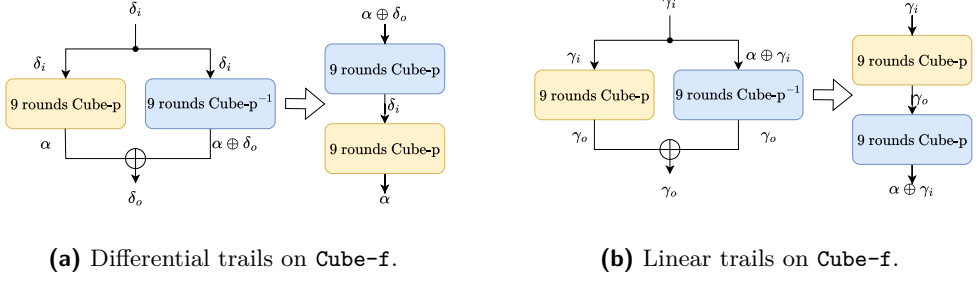


Figure 7: Trails on Cube-f from the ZIP construction perspective.

4.3.1 Collision attacks

Suppose there are two different messages P and Q that produce an inner collision. Then a state collision can be obtained from $P\|A, Q\|B$, for any messages A and B satisfying $\text{absorb}(P) \oplus A = \text{absorb}(Q) \oplus B$. Then, for any suffix M , the inputs $P\|A\|M$ and $Q\|B\|M$ produce an output collision, regardless of the digest length n .

In a random sponge, the expected workload to generate an inner collision is of the order $2^{(c+3)/2}$. By contrast, for a random oracle truncated to n bits, the expected workload of finding an output collision is of the order $2^{(n+3)/2}$. Hence, when $n < c$, a random sponge truncated to n bits provides the same resistance against output collisions as a random oracle truncated to n bits. If $n > c$, the best strategy to generate an output collision is to first find an inner collision. However, when $n < c$, using an inner collision does not reduce the expected workload. The dominant attack is instead the birthday attack with expected cost $2^{n/2}$.

Table 5: Workload of collision attacks on the proposed MasterCube instances.

	MasterCube-512	MasterCube-768	MasterCube-1024
theoretical workload	$2^{n/2}$	$2^{n/2}$	$2^{n/2}$
concrete workload	2^{256}	2^{384}	2^{512}

4.3.2 Preimage attacks

In the sponge construction, a preimage can be found by binding the target output string to an internal state and then finding a path to that state. However, given a random transformation f , once the output has been bound to a state s , we cannot compute a state $t = f^{-1}(s)$. Therefore, the output must be bound directly to the predecessor state t . Instead of guessing the inner part of the state corresponding with the first output block Z_0 , one has to guess a state t satisfying $f(t) = T_0$. This increases the number of trials by a factor 2^r . As a result, the expected workload becomes 2^n when $n < b$ and 2^b when $n > b$. The expected workload for finding a preimage is hence $2^n + 2^{c-1}$ for $n < b$ and $2^b + 2^{c-1}$ for $n > b$. Finally, when $n < c$, a truncated sponge function instantiated with a random transformation provides a similar resistance against preimages as a truncated random oracle.

4.3.3 Second preimage attacks

Assume that one wants to find a second preimage for a message M and let P be the padded version of M . In a sponge function, a second preimage is obtained if one can find

Table 6: Workload of preimage attacks on the proposed MasterCube instances.

	MasterCube-512	MasterCube-768	MasterCube-1024
theoretical workload	$2^n + 2^{c-1}$	$2^n + 2^{c-1}$	$2^n + 2^{c-1}$
concrete workload	2^{575}	2^{830}	2^{1087}

a second path to the inner state $t = \widehat{absorb}(P')$, where P' is the prefix of P obtained by removing only its last block $P_{|P|}$. Given such a path, one can construct a message $Q||X$ such that $absorb(Q||X) = absorb(P)$ where $X = \overline{absorb}(Q) \oplus \overline{absorb}(P') \oplus P_{|P|}$. For a random transformation f , the expected workload of this problem is $\frac{2^c}{|P|}$ provided that $|P| < 2^{c/2}$. This expected workload must be at least that of generating an inner collision as any second preimage induces an inner collision.

In a truncated random oracle, the expected workload for finding a second preimage is on the order of 2^n , independent of $|P|$. Hence, given the upper bound on the maximum number of message blocks $|P|$, a sponge function instantiated with a random transformation offers a similar level of resistance against second preimages as a truncated random oracle whenever $n < c - \log_2(|P|)$.

Table 7: Workload of second preimage attacks on the proposed MasterCube instances.

	MasterCube-512	MasterCube-768	MasterCube-1024
theoretical workload	$\frac{2^c}{ P }$	$\frac{2^c}{ P }$	$\frac{2^c}{ P }$
concrete workload	$2^{538.5}$	$2^{794.2}$	$2^{1049.7}$

4.4 Indifferentiability

The indifferentiability of the sponge construction with a random transformation has been proven by Bertoni et al. [4]. The sponge construction is subject to an indifferentiability security bound of $c/2$ bits, where c denotes the capacity. The advantage of an adversary $\mathcal{A}$ in distinguishing the sponge construction from a random oracle is upper bound by $O(q^2 \cdot n \cdot 2^{-c})$, where q is the number of queries made by $\mathcal{A}$ to the sponge construction and its underlying transformation, and n is the output length of the sponge construction. Therefore, if the underlying transformation is secure, MasterCube is indifferentiable from a random oracle up to $2^{c/2}$ queries.

4.5 Length extension attack

The length extension attack is given a message M and its hash value $H(M)$, an adversary can produce a valid hash for a modified message $M' = M||X$. For a sponge function, length extension is successful if one can find the inner state at the end of the squeezing of M . This comes down to state recovery [5]. For a sponge-based hash function, the expected number of solutions of output binding is 2^{b-n} . In that case, the probability of success of length extension is $\max(2^{n-b}, 2^{-n})$. Therefore, the success probability of length extension is negligible for MasterCube, which is illustrated in Table 8.

4.6 Quantum security analysis

It was shown by Unruh [21] that the security of a hash function is weaker than expected against quantum adversaries. Although the security level of a hash function can be improved by increasing the output size (or the capacity in the sponge construction), it

Table 8: Length extension attack security for the proposed MasterCube instances.

	MasterCube-512	MasterCube-768	MasterCube-1024
$b = \text{rate} + \text{capacity}$	1536	1536	1536
hash size n	512	768	1024
Length extension attack	2^{-512}	2^{-768}	2^{-512}

is still important to analyze the security of MasterCube against quantum adversaries. In particular, Unruh [21] presented a strengthened security definition for hash functions in the quantum setting, which is named collapsing hash functions. We follow the same path as the collapsing sponge [10, 22] to analyze the post-quantum security of MasterCube. First, we recall the definition of collapsing hash functions and its relation to zero-preimage resistance.

Theorem 1. [22] *If the block function f is collapsing when restricted to the left and right halves of its output, respectively, and if it is hard to find a zero-preimage of f (restricted to the right half of its output), then the sponge construction based on f is also collapsing.*

It has been proven by Unruh [21] that the collapsing property directly implies the collision resistance in the quantum setting. Therefore, the security of MasterCube against quantum adversaries can be analyzed by considering the security of the underlying permutation Cube-p.

Definition 2 (zero-preimage resistance). [10, 22] We call a function $f^o : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is zero-preimage resistant iff any quantum-polynomial-time adversary A^o , we have that $\Pr[f^o(x) = 0^m : x \leftarrow A^o]$ is negligible.

Since Cube-p is an efficient invertible permutation, we do not directly use it for constructing MasterCube. A PRP-PRF switch Cube-f is built from the ZIP construction, which can also be treated as the XOR of two PRPs. Consequently, MasterCube can obtain the collapsing property if Cube-f is a collapsing function and zero-preimage resistant. We then analyze the security of Cube-f as follows.

Theorem 2. *Cube-f is zero-preimage resistant.*

Proof. We can treat Cube-f as the XOR of two independent 512-bit PRPs, each of which is built from the 9-round Cube-p. To obtain a zero preimage of Cube-f, the adversary needs to find an output y , which implies $\text{Cube-p}(y) \oplus \text{Cube-p}^{-1}(y) = 0$. According to the security of Cube-f in the ZIP construction, the probability of such an event can be reduced to the differential attack of the full-round Cube-p. Therefore, the probability of finding a zero-preimage of Cube-f is at most the probability of a differential characteristic of Cube-p, which is negligible. $\square$

According to the definition given by [10] and the above theorem, we can conclude that Cube-f is a collapsing function. Then, we can further conclude that MasterCube is also a collapsing function based on the following lemma.

Lemma 1. *If Cube-f is a collapsing function, then MasterCube is also a collapsing function.*

For preimage and second-preimage resistance, we follow the quantum security analysis of the sponge construction by Alagic et al. [2]. First we recall the indistinguishability of the sponge construction in the quantum setting.

Theorem 3. [2] Suppose that the sponge construction with a single round of squeezing and no constraint on the inputs, $\mathbf{Sp}^\phi : (\{0, 1\}^r)^* \rightarrow \{0, 1\}^r$, is indifferntiable from a random oracle $f : (\{0, 1\}^r)^* \rightarrow \{0, 1\}^r$. Then, the full sponge construction with arbitrary squeezing and a valid pad function is indifferntiable from a random oracle $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$.

Since **MasterCube** is a sponge construction based on **Cube-f**, we can conclude that **MasterCube** is quantum indifferntiable from a random oracle if the single round sponge construction with **Cube-f** is quantum indifferntiable from a random oracle. Therefore, we can conclude that **MasterCube** is (second) preimage and collision resistant against quantum adversaries. The quantum lower bounds of **MasterCube** can be directly derived from the following results from [2].

Corollary 1 (quantum collision resistance). *The probability that a quantum algorithm $\mathcal{A}$ with quantum query access to a random function $\mathbf{Cube-f} \in S_{\{0,1\}^{r+c}}$ and its inverse, making at most a total of q queries, returns $m, m' \in (\{0, 1\}^r)^{\leq l}$ for $l \leq q$ such that $m = m'$ and $\mathbf{MasterCube}(m) = \mathbf{MasterCube}(m')$, can be upper bounded as*

$$Pr_{m, m' \leftarrow \mathcal{A}^{\mathbf{Cube-f}, \mathbf{Cube-f}^{-1}}} [\mathbf{MasterCube}(m) = \mathbf{MasterCube}(m')] \leq O(q^5 \cdot n \cdot 2^{-\min(r, c)}).$$

Corollary 2 (quantum preimage resistance). *Given $y \in \{0, 1\}^r$, the probability that a quantum algorithm $\mathcal{A}$ with quantum query access to a random function $\mathbf{Cube-f} \in S_{\{0,1\}^{r+c}}$ and its inverse, making at most a total of q queries, returns $m \in (\{0, 1\}^r)^{\leq l}$ for $l \leq q$ such that $y = \mathbf{MasterCube}(m)$, can be upper bounded as*

$$Pr_{m \leftarrow \mathcal{A}^{\mathbf{Cube-f}, \mathbf{Cube-f}^{-1}}} [y = \mathbf{MasterCube}(m)] \leq O(q^5 \cdot n \cdot 2^{-\min(r, c)}).$$

4.7 Statistical randomness tests

To evaluate the statistical randomness of the proposed hash function outputs, we applied both the NIST Statistical Test Suite (STS), SP 800-22 Rev.1a [3], and the Chinese randomness testing specification GM/T 0005-2021 [1]. In all experiments, binary sequences were generated by concatenating successive hash digests until the target sequence length was reached. Each sample sequence was 2^{20} bits long, and all sequences were tested independently.

Since the statistical behavior of a hash function may depend on the structure of its inputs, we considered five message-generation modes: random messages, counter messages, output-feedback (OFB) messages, low-Hamming-weight messages, and high-Hamming-weight messages. For each mode, we generated binary sequences of length 2^{20} bits and evaluated them separately. The same data-generation procedure and sequence length were used for both **MasterCube** and the standard hash function SHA-3. We note that each input message had a fixed length of input bits as the digest bits.

For the NIST STS tests, we adopted $m = 100$ sequences per configuration with significance level $\alpha = 0.01$. We report two metrics for each test item: the proportion of sequences that passed the test and the p -value measuring the uniformity of the distribution of p -values across all tested sequences. Following the standard NIST interpretation rule, a test result is regarded as acceptable when the pass proportion lies within the confidence interval

$$\hat{p} \pm 3\sqrt{\frac{\hat{p}(1-\hat{p})}{m}}, \quad \hat{p} = 1 - \alpha,$$

where $\alpha = 0.01$, yielding a minimum pass threshold of approximately 96/100 for $m = 100$ sequences. Additionally, the uniformity p -value must be not smaller than 0.0001.

For the Chinese GM/T 0005-2021 tests, we adopted the standard-prescribed significance level $\alpha = 0.01$. Each sample sequence of length 2^{20} bits was tested independently across 15 statistical test items. The acceptance threshold is that the p -value of each test must

satisfy $p \geq \alpha$. We tested 1000 independent samples per configuration to provide strong statistical evidence, resulting in 30,000 total samples (including each message generation mode and different instances of **MasterCube**) and 450,000 individual statistical tests.

We note that occasional failures of individual test instances may occur in statistical testing and do not necessarily indicate a structural weakness of the hash function. With significance level $\alpha = 0.01$, approximately 1% of tests would fail randomly even for a perfectly random source. Therefore, our analysis focuses on the overall passing percentage and the consistency of results across different message-generation modes, rather than on isolated failed cases. For reference, the same testing procedure was also applied to SHA-3 under identical settings.

Table 9: Randomness tests of **MasterCube** from different modes.

Standard	Mode	MasterCube-512	MasterCube-768	MasterCube-1024	SHA3-512
NIST SP 800-22					
NIST	Random	99.47%	99.47%	99.47%	100.00%
NIST	Counter	100.00%	100.00%	100.00%	100.00%
NIST	OFB	100.00%	98.94%	100.00%	99.47%
NIST	LowHW	99.47%	100.00%	100.00%	99.47%
NIST	HighHW	100.00%	100.00%	100.00%	99.47%
GM/T 0005-2021					
GM/T	Random	98.58%	98.96%	99.03%	98.63%
GM/T	Counter	98.48%	98.83%	98.98%	98.71%
GM/T	OFB	98.79%	98.79%	99.04%	99.04%
GM/T	LowHW	98.79%	98.90%	98.91%	98.90%
GM/T	HighHW	98.60%	98.79%	99.06%	98.77%

The experimental results show that **MasterCube** achieves stable statistical behavior under both NIST STS and GM/T 0005-2021. The randomness quality of **MasterCube** is comparable to that of the reference hash function SHA-3, achieving the same level of statistical randomness.

5 Implementations

According to the guidelines for performance self-assessment of cryptographic hash functions, we provide the software implementations of **MasterCube** in three categories: reference implementations, performance-optimized implementations, and resource-optimized implementations. The software implementations are evaluated on x86-64 and ARM platforms. To provide initial insights of the hardware performance of **MasterCube**, the preliminary hardware implementations are synthesized using a 45 nm Nangate standard cell library. The methods and performance results of these implementations are briefly described in this section.

5.1 Software implementations

5.1.1 Implementations on 64-bit platforms

The implementation of **MasterCube** follows Section 2, where each lane is represented by two `uint16_t` values in C programming language. Thus, the whole state is stored as a $6 \times 8 \times 2$ array of `uint16_t`. When SIMD instruction sets are available, the state can be stored in a more compact form. For SSE and NEON instruction sets, each row of a slice is stored as a 128-bit register. Then, the whole state takes 12 such registers, while the total numbers of available 128-bit registers are 16 for SSE and 32 for NEON. Thus, the whole state can be stored merely in registers, and there are still some free registers

for temporary values. For AVX2 instruction set, each slice is stored in 3 256-bit registers, where two consecutive rows are in the same register. Operations are then applied to 128-bit or 256-bit registers. When AVX512 is available, we can use the optimized instruction called TernaryLogic. It allows to implement several operations on 3 inputs with a single instruction, such as XORs of 3 values and combinations of AND and XOR in the MAndRX operation. MasterCube has many linear operations that XORs 3 values, and thus it can be optimized by a single TernaryLogic.

We evaluate the performance of the proposed MasterCube implementations on both of x86-64 and ARM platforms. The x86-64 experiments were conducted on an Intel Xeon Gold 6278C processor running at 2.60 GHz, equipped with 16 physical cores and 32 threads. The machine runs Ubuntu 22.04 LTS with Linux kernel 5.15.0-157-generic. The implementations were compiled with GCC 11.4.0 on X86-64 platform.

We also report results on ARM platform, namely a Raspberry Pi 5 Model B Rev. 1.0. The processor is an AArch64 64-bit ARM CPU with four physical cores and four logical threads, and its highest available SIMD instruction set is NEON. The system runs Debian GNU/Linux 12 (bookworm) with Linux kernel 6.12.47+rpt-rpi-2712. The ARM implementations were compiled with GCC 12.2.0 together with the `sse2neon` [18] compatibility layer for the NEON version. Cycle counts on the ARM platform are obtained using the Linux `perf_event` CPU-cycle counter. The implementations on both x86-64 and ARM platforms are compiled with the settings in Table 10.

Table 10: Implementation platforms and GCC compilation commands for MasterCube.

Architecture	Platform	Implementation Type	GCC Compile Code
x86-64	Intel Xeon Gold 6287C Ubuntu 22.04 LTS with Linux kernel 5.15.0-157-generic GCC 11.4.0	Reference	<code>gcc -std=c99 -Wpedantic -Wall -Wextra -O2</code>
		Performance-optimized	<code>gcc -O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra</code>
		Resource-optimized	<code>gcc -Os -march=x86-64 -mavx2 -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra</code>
ARM-64	Raspberry Pi 5 Model B Rev. 1.0. AArch64 64-bit ARM CPU Debian GNU/Linux 12 (bookworm) with Linux kernel 6.12.47+rpt-rpi-2712. GCC 12.2.0	Reference	<code>gcc -std=c99 -Wpedantic -Wall -Wextra -O2</code>
		Performance-optimized	<code>gcc -O3 -march=armv8.2-a+sve -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra</code>
		Resource-optimized	<code>gcc -Os -march=armv8-a -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra</code>

All benchmarks are single-message tests. For each algorithm and implementation, we measure the cost of hashing one message of a fixed length and report the average number of CPU cycles per byte. Multiple block sizes are used to capture both short-message overhead and long-message steady-state performance. A deterministic seed is used in all tests to make the benchmark inputs reproducible. The results are summarized in Table 11.

5.1.2 Implementations on 32-bit platforms

For 32-bit platforms, we focus on 32-bit Arm CPUs. The implementation has two variants, depending on the availability of the NEON instruction set. When NEON is not available, the implementation falls back to almost the same approach used on 64-bit platforms without SIMD. The only difference is in data copying. 64-bit platforms can copy 64-bit values as a unit, whereas 32-bit platforms can copy only 32-bit values at a time. When NEON is available, it is identical to the 64-bit ARM implementation with NEON.

Table 11: Performance of MasterCube on x86-64 and ARM platforms under the self-assessment guidelines. All results are reported in CPU cycles per byte.

Platform	Algorithm	Impl.	32B	128B	512B	1024B	4096B	8192B	16384B	65536B
x86-64	MasterCube-512	Reference	546.48	272.36	168.77	151.57	147.13	144.91	144.74	144.45
	MasterCube-512	Performance-optimized	46.55	21.05	12.46	11.05	10.60	10.39	10.32	10.36
	MasterCube-512	Resource-optimized	54.93	24.69	14.65	12.93	12.32	12.14	11.99	11.71
	MasterCube-512	AVX512	80.57	28.72	13.98	11.52	10.22	9.91	9.76	9.69
	MasterCube-768	Reference	1040.58	392.11	226.94	210.12	193.95	191.30	193.70	192.36
	MasterCube-768	Performance-optimized	88.47	31.63	17.55	16.04	14.55	14.35	14.17	14.10
	MasterCube-768	Resource-optimized	100.14	35.86	19.93	18.16	16.53	16.23	16.07	15.96
	MasterCube-768	AVX512	81.59	29.22	16.21	14.79	13.84	13.47	13.20	13.01
	MasterCube-1024	Reference	1963.79	813.81	481.73	403.75	361.82	354.49	350.57	348.49
	MasterCube-1024	Performance-optimized	132.16	52.52	30.36	26.26	23.50	22.95	22.74	22.58
	MasterCube-1024	Resource-optimized	145.36	58.04	33.58	29.06	25.99	25.42	25.20	25.07
	MasterCube-1024	AVX512	120.16	47.64	27.53	23.78	21.73	21.04	20.68	20.43
ARM	MasterCube-512	Reference	215.62	108.12	67.81	61.07	59.46	58.61	58.20	58.10
	MasterCube-512	Performance-optimized	111.89	57.71	35.80	32.18	31.21	30.75	30.56	30.47
	MasterCube-512	Resource-optimized	396.85	412.89	257.94	231.78	225.31	222.04	220.64	220.04
	MasterCube-768	Reference	431.48	161.00	93.88	86.91	80.09	79.24	78.39	77.80
	MasterCube-768	Performance-optimized	224.94	85.29	50.44	47.17	43.59	43.16	42.72	42.42
	MasterCube-768	Resource-optimized	1346.36	504.64	294.08	273.19	251.95	249.40	246.64	244.69
	MasterCube-1024	Reference	645.97	268.43	160.52	140.34	126.86	124.34	123.08	122.56
	MasterCube-1024	Performance-optimized	343.27	143.94	86.81	76.11	68.96	67.60	66.93	66.54
	MasterCube-1024	Resource-optimized	2467.05	1028.07	616.22	538.73	488.05	478.39	473.52	470.88

5.1.3 Performance comparison of software implementations

For comparison, we use SHA3-512 from the Keccak Code Package (XKCP [23]), evaluated under the corresponding scalar, AVX2, and AVX512 implementations. In addition, we also include the SHA3-512 implementation from OpenSSL 4.1.0-dev [19], built locally from the master branch. For compatibility reasons, this OpenSSL version was built for the `linux-x86_64` platform with GCC using the following main optimization flags: `-O3 -DL_ENDIAN -DOPENSSL_PIC -DNDEBUG`. This provides an additional reference point against a highly optimized general-purpose cryptographic library.

On x86-64 platform, we evaluate the implementations of MasterCube-512 based on reference, AVX2 (performance-optimized) and AVX512. The implementations were compiled with GCC 11.4.0 using the `-O3` optimization level.

The ARM implementations were compiled with GCC 12.2.0 using `-std=c99 -Wall -Wextra -O3 -march=armv8-a+simd` together with the `sse2neon` [18] compatibility layer for the NEON version. Cycle counts on the ARM platform were obtained using the Linux `perf_event` CPU-cycle counter.

All benchmarks are single-message tests. For each algorithm and implementation, we measure the cost of hashing one message of a fixed length and report the average number of CPU cycles per byte. Multiple block sizes are used to capture both short-message overhead and long-message steady-state performance. A deterministic seed is used in all tests to make the benchmark inputs reproducible. The results are summarized in Table 12.

As shown in Table 12, SIMD optimizations substantially improve the performance of MasterCube on both platforms. On the x86-64 platform, the AVX2/AVX512 implementation of MasterCube-512 achieves better steady-state performance than the SHA3-512 for messages longer than 512 bytes. These results suggest that the proposed 512-bit variant has a favorable performance profile on x86-64 platforms with AVX2/AVX512 support. Compares with Table 11, the performance of MasterCube-768 and MasterCube-1024 is also competitive with SHA3-512, especially for long messages. We note that the performance of XKCP’s SHA3-512 implementation is highly optimized with hand-tuned assembly code, while our MasterCube implementations are written in C and compiled with standard optimization flags. This indicates that further performance improvements for MasterCube are possible through low-level optimizations and architecture-specific tuning.

Table 12: Performance comparison of MasterCube-512 and SHA3-512 on x86-64 and ARM platforms. All results are reported in CPU cycles per byte.

Platform	Algorithm	Impl.	32B	128B	512B	1024B	4096B	8192B	16384B	65536B
x86-64	MasterCube-512	Reference	546.48	272.36	168.77	151.57	147.13	144.91	144.74	144.45
	MasterCube-512	Performance-optimized	46.55	21.05	12.46	11.05	10.60	10.39	10.32	10.36
	MasterCube-512	AVX512	80.57	28.72	13.98	11.52	10.22	9.91	9.76	9.69
	SHA3-512	XKCP Scalar	41.19	20.46	19.62	18.21	18.85	17.21	17.22	17.36
	SHA3-512	XKCP AVX2	27.69	13.70	13.53	12.51	12.00	11.91	11.91	11.90
	SHA3-512	XKCP AVX512	24.28	12.05	11.72	10.93	10.39	10.35	10.37	10.35
	SHA3-512	OpenSSL AVX512	76.57	26.89	18.51	16.15	14.33	14.15	14.06	14.01
ARM	MasterCube-512	Reference	215.62	108.12	67.81	61.07	59.46	58.61	58.20	58.10
	MasterCube-512	Performance-optimized	111.89	57.71	35.80	32.18	31.21	30.75	30.56	30.47
	SHA3-512	XKCP Plain	68.55	35.08	34.94	32.85	31.29	31.28	31.28	31.33
	SHA3-512	XKCP NEON	66.52	33.02	32.64	30.56	28.98	28.98	28.98	28.95

On the ARM platform, the NEON implementation also reduces the costs of MasterCube compared with the plain implementation. However, the XKCP NEON implementation of SHA3-512 remains faster on this platform. This is expected because the current ARM implementation of MasterCube is obtained through the `sse2neon` compatibility layer rather than a hand-written assembly NEON implementation. Therefore, further ARM-specific optimization remains a natural direction for improving the portability and efficiency of MasterCube.

5.2 Hardware implementations

Two drafts of the MasterCube hardware implementation were designed: an area-optimized implementation and a performance-optimized implementation. Hardware synthesis and verification were performed under a unified environment, as summarized in Table 13. The target operating frequency for all implementations was set to 100 MHz. All synthesis, timing, area, and power reports were generated following the procedure described in Algorithm 5. We emphasize that our hardware implementations are preliminary and the performance results are roughly gathered. Further optimization and exploration of the design space are needed to fully understand the hardware performance of MasterCube. The current results are intended to provide initial insights into the potential hardware efficiency of MasterCube and to serve as a reference point for future improvements.

Table 13: Hardware synthesis environment.

Item	Configuration
Synthesis Tool	Synopsys Design Compiler 2016
Operating System	CentOS 7 (64-bit)
Technology	Nangate 45 nm library
Target Frequency	100 MHz

We developed two preliminary hardware implementations for MasterCube-512/768/1024 to evaluate the feasibility of the proposed permutation architecture. These implementations are not yet the final resource-prioritized and performance-prioritized versions required by the algorithm self-evaluation process. In the next stage, we will further refine the architectures according to the evaluation requirements, complete the missing metrics, and separately optimize the design toward resource efficiency and high throughput.

The first implementation is a round-sequential architecture. It adopts a command-based control interface, where the top-level FSM manages input absorption, permutation invocation, and output reading. The permutation engine evaluates one complete forward or inverse round combinationally in each clock cycle. During permutation, the current

Algorithm 5 Hardware synthesis flow for MasterCube.

```

1: procedure SYNTHESIZE MASTERCUBE
2:   target_library  $\leftarrow$  "/45nmNangate.db" ▷ Specify target library
3:   read_verilog ▷ Read Verilog design files
4:   set_auto_wire_load_selection false
5:   create_clock -period 10 -name clk [get_ports clk]
6:   set_dont_touch_network [get_ports clk] ▷ Preserve clock network
7:   set_input_delay -clock clk -max 0.5 [all_inputs]
8:   set_input_delay -clock clk -min 0.1 [all_inputs]
9:   set_output_delay -clock clk -max 0.5 [all_outputs]
10:  set_output_delay -clock clk -min 0.1 [all_outputs] ▷ Timing constraints
11:  set_max_transition 2 [current_design]
12:  set_max_fanout 20 [current_design]
13:  set_max_area 0 ▷ No area constraint
14:  compile_ultra ▷ Perform synthesis
15:  report_area  $\rightarrow$  "/area.rpt"
16:  report_timing  $\rightarrow$  "/timing.rpt"
17:  report_power  $\rightarrow$  "/power.rpt" ▷ Generate reports
18: end procedure

```

1536-bit state is copied into an auxiliary state register. The forward path first executes rounds 0–8 on the main state, and the inverse path then executes rounds 17–9 on the auxiliary state. In the last round state, the inverse result is XORed with the forward result to generate the updated state. Thus, this implementation is sequential at the round-scheduling level, while each round is still computed over the full 1536-bit state.

Table 14: Round-sequential implementation synthesis results of MasterCube.

Design	Area (μm^2)	GE	Delay (ns)	Power (μW)	Cycles per 32B	
					Absorb	Squeeze
MasterCube-512					67	0
MasterCube-768	94,713	118,689	1.54	10,122	55	20
MasterCube-1024					43	40

Note: NAND area = $0.798 \mu\text{m}^2$ (45 nm Nangate library).

The cycle counts of the round-sequential implementation are calculated according to the top-level FSM. For one absorbed block, the clear command requires 2 cycles, each 64-bit input word requires 3 cycles, and the permutation call requires 20 cycles, including command issue, 18 round-computation cycles, and one completion-wait cycle. Since the absorbed rate block contains 15, 11, and 7 64-bit words for MasterCube-512, MasterCube-768, and MasterCube-1024, respectively, the total cycle counts are $2 + 15 \times 3 + 20 = 67$, $2 + 11 \times 3 + 20 = 55$, and $2 + 7 \times 3 + 20 = 43$ cycles per absorbed block. The preliminary synthesis results of this implementation are summarized in Table 14.

The second implementation is a parallel phase-sliced architecture. It follows a 64-bit streaming absorption flow, where each 64-bit input word is XORed into the rate portion of the 1536-bit internal state. Once a rate block is filled, or when the final padded block is formed, the permutation core is invoked. In this implementation, the 1536-bit state is duplicated into two branches. The forward branch applies rounds 0–8, while the inverse branch applies rounds 17–9. Each round is divided into two sequential combinational phases over the full state, and the two branches are updated in parallel. Therefore, the round computation requires $9 \times 2 = 18$ cycles, followed by one additional cycle to XOR

the forward and inverse branch results. Since the absorbed rate block contains 15, 11, and 7 64-bit words for MasterCube-512, MasterCube-768, and MasterCube-1024, respectively, the total cycle counts are $15 + 19 = 34$, $11 + 19 = 30$, and $7 + 19 = 26$ cycles per absorbed block. The preliminary synthesis results of this implementation are summarized in Table 15.

Table 15: Parallel phase-sliced implementation synthesis results of MasterCube.

Design	Area (μm^2)	GE	Delay (ns)	Power (μW)	Cycles per 32B	
					Absorb	Squeeze
MasterCube-512					34	0
MasterCube-768	102,798	128,821	1.66	55,559	30	19
MasterCube-1024					26	26

Note: NAND area = $0.798 \mu\text{m}^2$ (45 nm Nangate library).

From the current implementations, the main hardware-area bottlenecks of MasterCube come from three aspects. First, the ZIP structure requires both forward and inverse computation states, which increases register usage and state-management logic. Second, the round function operates on the complete 1536-bit state and contains multiple MAndRX operations and wide linear mixing networks. Third, the current designs still include full-state multiplexing and control logic for branch selection, phase scheduling, command execution, and state update. These parts dominate the preliminary area results and will be the main targets for subsequent optimization, including finer-grained datapath reuse, reduction of full-state multiplexers, and dedicated resource-prioritized/performance-prioritized architecture refinement.

5.3 Tensor-based implementation of keyed Cube-f

The security of MasterCube in keyed scenarios, such as Message Authentication Codes (MAC) and Authenticated Encryption with Associated Data (AEAD), relies fundamentally on the protection of the secret key during the execution of Cube-f. We propose a tensor-based white-box implementation for the keyed variant of Cube-f that supports two primary modes as follows.

- **MAC Mode (Key as State Input):** Following the construction of KMAC [15], the secret key K is padded and prepended to the message, effectively becoming a part of the initial state S_0 . In this mode, the white-box implementation must ensure that the key-dependent information remains obfuscated throughout the initial absorbing phases.
- **AEAD Mode (Key as Branch XOR or State Input):** In AEAD constructions, the key can either be treated as a static state input or be periodically XORed into specific branches of the ZIP construction. The implementation provides the flexibility to bind these key-dependent constants directly into the obfuscated transformation logic.

The fundamental construction of the white-box keyed Cube-f leverages a combination of linear matrix encodings and tensor-product linearization. Each m -bit lane X of the state is protected by a secret linear encoding matrix $\mathbf{M}$. To resolve the non-linearity introduced by the logical-AND operations within the AndRX structure, we project the state into a higher-dimensional space, which is the tensor product of the input state with itself ($X \otimes X \in \mathbb{F}_2^{m^2}$). By formulating the round function as a quadratic mapping, the

implementation evaluates each round as a single matrix-vector multiplication in the tensor space as

$$\tilde{X}^{(i+1)} = \tilde{\mathbf{Q}}_i \cdot (\tilde{X}^{(i)} \otimes \tilde{X}^{(i)}),$$

where the obfuscated round matrix $\tilde{\mathbf{Q}}_i$ incorporates the input decoding $(\mathbf{M}^{(i)} \otimes \mathbf{M}^{(i)})^{-1}$, the round functions (contain the AndRX structure), and the output encoding $\mathbf{M}^{(i+1)}$. This algebraic approach allows the keyed **Cube-f** to be executed with high efficiency while providing structural resistance against key extraction. Detailed mathematical formulations of the encoding construction, formal security proofs against ASA decomposition, and comprehensive performance benchmarks are provided in **Appendix B**.

References

- [1] Randomness test specification, 2021-10-18 2021. <https://www.chinesestandard.net/PDF.aspx/GMT0005-2021>.
- [2] Gorjan Alagic, Joseph Carolan, Christian Majenz, and Saliha Tokat. The sponge is quantum indifferentiable. In *66th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2025, Sydney, Australia, December 14-17, 2025*, pages 2591–2630. IEEE, 2025.
- [3] Lawrence Bassham, Andrew Rukhin, Juan Soto, James Nechvatal, Miles Smid, Stefan Leigh, M Levenson, M Vangel, Nathanael Heckert, and D Banks. A statistical test suite for random and pseudorandom number generators for cryptographic applications, 2010-09-16 2010. Accessed: 2026-04-29.
- [4] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the sponge construction. In Nigel P. Smart, editor, *Advances in Cryptology - EUROCRYPT 2008, 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Istanbul, Turkey, April 13-17, 2008. Proceedings*, Lecture Notes in Computer Science, pages 181–197. Springer, 2008.
- [5] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Cryptographic sponge functions, 2011. <https://keccak.team/files/CSF-0.1.pdf>.
- [6] Alex Biryukov and Adi Shamir. Structural cryptanalysis of SASAS. In Birgit Pfitzmann, editor, *Advances in Cryptology - EUROCRYPT 2001, International Conference on the Theory and Application of Cryptographic Techniques, Innsbruck, Austria, May 6-10, 2001, Proceeding*, volume 2045 of *Lecture Notes in Computer Science*, pages 394–405. Springer, 2001.
- [7] Alex Biryukov and Adi Shamir. Structural cryptanalysis of SASAS. *Journal of Cryptology*, 23(4):505–518, 2010.
- [8] Jung Hee Cheon, Hyunsook Hong, Joohee Lee, and Jooyoung Lee. An efficient affine equivalence algorithm for multiple S-boxes and a structured affine layer. In Roberto Avanzi and Howard M. Heys, editors, *Selected Areas in Cryptography - SAC 2016 - 23rd International Conference, St. John's, NL, Canada, August 10-12, 2016, Revised Selected Papers*, volume 10532 of *Lecture Notes in Computer Science*, pages 299–316. Springer, 2016.
- [9] Stanley Chow, Philip A. Eisen, Harold Johnson, and Paul C. van Oorschot. White-box cryptography and an AES implementation. In Kaisa Nyberg and Howard M. Heys, editors, *Selected Areas in Cryptography, 9th Annual International Workshop, SAC 2002, St. John's, Newfoundland, Canada, August 15-16, 2002. Revised Papers*, Lecture Notes in Computer Science, pages 250–270. Springer, 2002.

- [10] Jan Czajkowski, Leon Groot Bruinderink, Andreas Hülsing, Christian Schaffner, and Dominique Unruh. Post-quantum security of the sponge construction. In Tanja Lange and Rainer Steinwandt, editors, *Post-Quantum Cryptography - 9th International Conference, PQCrypto 2018, Fort Lauderdale, FL, USA, April 9-11, 2018, Proceedings*, Lecture Notes in Computer Science, pages 185–204. Springer, 2018.
- [11] Patrick Derbez, Pierre-Alain Fouque, Baptiste Lambin, and Brice Minaud. On recovering affine encodings in white-box implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(3):121–149, 2018.
- [12] Itai Dinur. An improved affine equivalence algorithm for random permutations. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EURO-CRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part I*, volume 10820 of *Lecture Notes in Computer Science*, pages 413–442. Springer, 2018.
- [13] Antonio Flórez-Gutiérrez, Lorenzo Grassi, Gregor Leander, Ferdinand Sibleyras, and Yosuke Todo. General practical cryptanalysis of the sum of round-reduced block ciphers and ZIP-AES. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part IX*, Lecture Notes in Computer Science, pages 280–311. Springer, 2024.
- [14] Keccak Team. The keccak reference and specification. <https://keccak.team/keccak.html>, n.d. Accessed: 2026-04-29.
- [15] John Kelsey, Shu-jen Chang, and Ray Perlner. SHA-3 derived functions: cSHAKE, KMAC, TupleHash, and ParallelHash. *NIST special publication*, 800:185, 2016.
- [16] Stefan Kölbl, Gregor Leander, and Tyge Tiessen. Observations on the SIMON block cipher family. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I*, Lecture Notes in Computer Science, pages 161–185. Springer, 2015.
- [17] Jiqiang Lu, Zhigang Zhao, and Huaqun Guo. White-box implementation of the KMAC message authentication code. In Swee-Huay Heng and Javier López, editors, *Information Security Practice and Experience - 15th International Conference, ISPEC 2019, Kuala Lumpur, Malaysia, November 26-28, 2019, Proceedings*, Lecture Notes in Computer Science, pages 248–270. Springer, 2019.
- [18] sse2neon. <https://github.com/DLTCollab/sse2neon/tree/3b70b3727edc9a151c113814129258c3423a771c>. Accessed: 2026-04-29.
- [19] OpenSSL Project. <https://github.com/openssl/openssl/tree/95b31c03358962a2e03ae414f5d4e0925b553959>. Accessed: 2026-04-29.
- [20] Laurent Perron and Frédéric Didier. CP-SAT (v9.9). <https://developers.google.com/optimization/cp/>.
- [21] Dominique Unruh. Collapse-binding quantum commitments without random oracles. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part II*, Lecture Notes in Computer Science, pages 166–195, 2016.

- [22] Dominique Unruh. Collapsing sponges: Post-quantum security of the sponge construction. *IACR Cryptol. ePrint Arch.*, 2017:282, 2017.
- [23] eXtended Keccak Code Package. <https://github.com/XKCP/XKCP/tree/716f007dd73ef28d357b8162173646be574ad1b7>. Accessed: 2026-04-29.

A Test Vectors for MasterCube

Below are the test vectors for the proposed MasterCube instances in this specification.

Listing 1: Test vectors.

```

=== MasterCube-512 ===
Length of the message = 16
Message = 5D9B
Hash value = C6ADAAAFAC38E7BDAA6EAF589376AACCE00D7DCA0B5AC9F6282C6082637B0196B42AB3C3C6D8F2D
A9D43E7B81FC17C3715CE5DDB7B0EA988AAC782925336B3FF7

Length of the message = 256
Message = 41BDB6A04FCD27406E77D0D2ADE868940824EC7124BDAF36CCE4AA0938A16BFC
Hash value = CAE91F74FAFE967DFA4796DF6A51231189BA1DFC71A03FF19F32DBDDE2CE0679D66030DCOC1810
FE48F5F5B8331D555A4DFCB7A849CC39F81944156E3AEDBD63

=== MasterCube-768 ===
Length of the message = 16
Message = 5D9B
Hash value = D0CBAAA4A32BC0D7C9CC9823AF87D257BF95FBBB024F6F8DFD74046B6A0B02C527FBB0304AA6B8
306E965D1413708AB466F96156E81B53F9C7C0B502AD5B777B9F06C6EE3E66091F5B10867D022C95A50E5
69FA184D05D2BC8AD55281AA2DA87

Length of the message = 256
Message = 41BDB6A04FCD27406E77D0D2ADE868940824EC7124BDAF36CCE4AA0938A16BFC
Hash value = 49F363B73F1221D05BE0FD575C2D588C67856A0A6CC5D35FA70B5712BEA84F6ED58D1FA696F686
86EAF716FE80387841D6016408D7884664325CDB3F453CC08C3B164CF30F9B713A99C3D5E381C6904E4
8CD65D13A457CB31C28C9D70EAD6F

=== MasterCube-1024 ===
Length of the message = 16
Message = 5D9B
Hash value = 1AA1713582BAB6838D0A4FA47CF6056D365D40ECF31EB8015E7CA0E14C0C5E606738934727667A
F0475E670D027D8915D4362693C69A43D72555255DE8BA4FD864DE95C3362711C2D319834FB73860A904C
1AF836DAB21D1D7B36A1849B5C4913B39B6284632246072AAA999F402A6D66C9CB984E73FAA3BE1136E2F
45859EA0

Length of the message = 256
Message = 41BDB6A04FCD27406E77D0D2ADE868940824EC7124BDAF36CCE4AA0938A16BFC
Hash value = 8AAE420D49F4C8B2538A3BBBF7A9768EA27701524D8BB124F1BF360F1243E6E8DD3E74342C3CE5
587EFD60565A26AF7DC156D020F9B51E5902932814C93EFED4C7423A88941E0248642FE48F9CA5C674877
5FB20A37050EFB9B3E983DFB48AB1E79D88D690F326D475B241452EFBFF4F7A565204535447E859B72275
B03F4740

```

B The tensor-based implementation of keyed Cube-f

B.1 White-box cryptography

The traditional black-box security model assumes that attackers only have access to the inputs and outputs of a cryptographic algorithm. However, in modern deployment scenarios, such as mobile applications, digital rights management (DRM), and software-based payment systems, cryptographic primitives are frequently executed in untrusted environments. The white-box attack model [9] assumes a worst-case scenario where the

adversary possesses full access to the software execution environment, including memory contents, execution flow, and intermediate registers. Under these conditions, the primary goal of white-box cryptography is to prevent the key extraction. The fundamental strategy is the application of encodings to the intermediate states of the cryptographic algorithm. In this framework, the original variables and operations are transformed into an encoded domain where the algebraic structure is hidden, while the functional correctness is preserved through a carefully designed network of bijections. Internal encodings are used to obfuscate the intermediate states between consecutive rounds of the algorithm. Let F_i denote the i -th round function of the **Cube-f**. To protect this function, we select a sequence of random bijective mappings $\{E_i\}$, referred to as internal encodings. The original round function F_i is then transformed into an obfuscated round function F'_i as follows.

$$F'_i = E_{i+1} \circ F_i \circ E_i^{-1}$$

The significance of this construction lies in its concatenation property. When multiple obfuscated rounds are executed sequentially, the output encoding of one round and the input encoding of the subsequent round are designed to cancel each other out.

$$\begin{aligned} F'_n \circ \dots \circ F'_1 &= (E_{n+1} \circ F_n \circ E_n^{-1}) \circ \dots \circ (E_2 \circ F_1 \circ E_1^{-1}) \\ &= E_{n+1} \circ (F_n \circ \dots \circ F_1) \circ E_1^{-1} \end{aligned}$$

This cancellation mechanism ensures that the underlying state follows the standard algorithm during the transition between rounds. To ensure that the entire white-box implementation remains secure, the external encodings are introduced at the boundaries of the algorithm. The input encoding $I = E_1$ and the output encoding $O = E_{n+1}$ protect the initial input and the final result of the execution.

B.2 Algebraic formulation of encoded lane operations

To protect the keyed **Cube-f** in a white-box implementation, it is imperative to securely execute both the non-linear AND-RX operations on individual lanes and the linear mixing between lanes. The white-box implementation employs a tensor-based framework with linear encodings.

Linear encoding. All the intermediate values within the **Cube-f** execution are consistently protected using linear encodings. Specifically, an original 32-bit lane $X \in \mathbb{F}_2^{32}$ is encoded as $\tilde{X}$, defined by

$$\tilde{X} = E(X) = \mathbf{M} \cdot X,$$

where $\mathbf{M}$ is a randomly chosen 32×32 invertible matrix over $\mathbb{F}_2$ (serving as a linear mixing bijection). All cryptographic operations within the round function are mapped algebraically into these encoded domains and executed under the encodings, ensuring the true state X never appears in memory.

Tensor framework for AND-RX structure. The most challenging component of the **Cube-f** to obfuscate is the MAndRX operation, as the logical-AND ($\wedge$) introduces non-linearity. However, instead of relying on inefficient and memory-consuming Lookup Tables (LUTs), we exploit the tensor product (Kronecker product) space to fully algebraicize the execution. The MAndRX transformation is an m -bit Feistel network utilizing bitwise shift ($\gg$), XOR ($\oplus$), and AND ($\wedge$) operations. Over the Boolean field $\mathbb{F}_2$, the AND operation between two bits x_i and x_j is simply a degree-2 multiplication: $x_i \cdot x_j$.

To linearize this quadratic operation, we construct the tensor product of the input state with itself, denoted as $X \otimes X \in \mathbb{F}_2^{m^2}$. This extended vector contains all possible degree-2

combinations of the bits of X . Furthermore, since $x_i^2 = x_i$ for all $x_i \in \mathbb{F}_2$, the diagonal entries of $X \otimes X$ perfectly encapsulate the degree-1 (linear) terms. Consequently, the entire m -bit MAndRX round function, encompassing all shifts, XORs, and logical-ANDs, can be evaluated simultaneously as a pure linear mapping from the m^2 -dimensional tensor space back to the m -dimensional lane space. Let the round function of round i be F_i . There exists a unique binary matrix $\mathbf{Q}_i \in \mathbb{F}_2^{m \times m^2}$ such that the state update is expressed entirely via matrix multiplication as

$$X^{(i+1)} = F_i(X^{(i)}) = \mathbf{Q}_i \cdot (X^{(i)} \otimes X^{(i)}).$$

To enhance the encoding diversity, we apply a sequence of various secret encoding matrices $\{\mathbf{M}^{(i)}\}$ at each round boundary. The encoded input for round i is $\tilde{X}^{(i)} = \mathbf{M}^{(i)} \cdot X^{(i)}$. By invoking the mixed-product property of the tensor product, the tensor squared of the encoded state can be written as

$$\tilde{X}^{(i)} \otimes \tilde{X}^{(i)} = (\mathbf{M}^{(i)} \cdot X^{(i)}) \otimes (\mathbf{M}^{(i)} \cdot X^{(i)}) = (\mathbf{M}^{(i)} \otimes \mathbf{M}^{(i)}) \cdot (X^{(i)} \otimes X^{(i)}).$$

Thus, the tensor product $X^{(i)} \otimes X^{(i)}$ can be recovered implicitly by applying the inverse tensor encoding

$$X^{(i)} \otimes X^{(i)} = (\mathbf{M}^{(i)} \otimes \mathbf{M}^{(i)})^{-1} \cdot (\tilde{X}^{(i)} \otimes \tilde{X}^{(i)}).$$

By substituting this into the MAndRX round function and applying the target output encoding $\mathbf{M}^{(i+1)}$ for the current round, the encoded state for the next round is evaluated as

$$\begin{aligned} \tilde{X}^{(i+1)} &= \mathbf{M}^{(i+1)} \cdot X^{(i+1)} \\ &= \mathbf{M}^{(i+1)} \cdot \mathbf{Q}_i \cdot \left[(\mathbf{M}^{(i)} \otimes \mathbf{M}^{(i)})^{-1} \cdot (\tilde{X}^{(i)} \otimes \tilde{X}^{(i)}) \right]. \end{aligned}$$

The matrix multiplication then combines the input decoding, the cryptographic round function, and the output encoding into a single obfuscated round matrix $\tilde{\mathbf{Q}}_i \in \mathbb{F}_2^{m \times m^2}$ as

$$\tilde{\mathbf{Q}}_i = \mathbf{M}^{(i+1)} \cdot \mathbf{Q}_i \cdot (\mathbf{M}^{(i)} \otimes \mathbf{M}^{(i)})^{-1}.$$

The white-box implementation merely computes the tensor product of the incoming obfuscated vector, $\tilde{X}^{(i)} \otimes \tilde{X}^{(i)}$, and performs a matrix-vector multiplication with $\tilde{\mathbf{Q}}_i$ as

$$\tilde{X}^{(i+1)} = \tilde{\mathbf{Q}}_i \cdot (\tilde{X}^{(i)} \otimes \tilde{X}^{(i)}).$$

This design ensures that the underlying values and cryptographic structures are fundamentally hidden within the obfuscated linear algebra.

From cell-level to lane-level linear operations. In the design of Cube-f, the linear diffusion layers (such as operations on sheets) are performed at the granularity of 16-bit cells, which represent the two halves of a lane. Let a 32-bit lane be denoted as $O = (L, R)$, where L and R are 16-bit cells. When a 32-bit linear encoding $\mathbf{M}$ is applied to protect the entire lane as $\tilde{O} = \mathbf{M} \cdot O$, the algebraic boundary between L and R is completely obfuscated. Consequently, it is impossible to directly extract or manipulate a single 16-bit cell within the encoded domain without compromising the encoding.

To resolve this, we demonstrate that any arbitrary linear operation over $\mathbb{F}_2$ on the 16-bit cells can be securely reduced to a sequence of permitted operations on the fully encoded 32-bit lanes. The white-box implementation on the AndRX structure fundamentally permits only two types of linear operations:

1. **Intra-lane linear operations:** Any arbitrary 32×32 linear transformation applied exclusively within a single lane $O = (L, R)$.

2. **Inter-lane full XOR operations:** The bitwise addition of two entire 32-bit lanes, e.g., $O_0 \oplus O_1 = (L_0 \oplus L_1, R_0 \oplus R_1)$.

To prove that these two permitted operations are sufficient to synthesize any $\mathbb{F}_2$ matrix over the constituent cells, it suffices to show that we can isolate any 16-bit half from a complete lane. Suppose we are given a lane $O_0 = (L_0, R_0)$ and we wish to isolate the left cell to compute $O_2 = (L_0, 0)$. This isolation is achieved through the following sequence of operations.

First, we apply an intra-lane linear operation to O_0 to compute

$$T_1 = (L_0 \oplus R_0, R_0).$$

Next, using the inter-lane full XOR operation, we add T_1 and the original O_0 as

$$T_2 = T_1 \oplus O_0 = (L_0 \oplus R_0 \oplus L_0, R_0 \oplus R_0) = (R_0, 0).$$

We then apply another intra-lane linear operation (a structural swap of the upper and lower halves) on T_2 to obtain

$$T_3 = (0, R_0).$$

Finally, by accumulating these intermediate results via inter-lane XOR operations, we successfully isolate the component L_0 , such that

$$O_2 = T_1 \oplus T_2 \oplus T_3 = (L_0 \oplus R_0, R_0) \oplus (R_0, 0) \oplus (0, R_0) = (L_0, 0).$$

Using symmetrical derivations, we can effectively compute and isolate $(0, R_0)$, $(L_0, 0)$, and $(R_0, 0)$ from any given lane. Having acquired the ability to isolate any specific half from any AndRX lane, we can subsequently add any half of any lane to any half of another lane. This constructively proves that any linear operation between all 16-bit halves across the state can be implemented exclusively using full 32-bit lane additions and intra-lane transformations. Therefore, the complex linear matrices on sheets can be fully accommodated within the white-box scheme.

Having established that all cell manipulations can be logically reduced to full 32-bit lane XORs and intra-lane operations, we now clarify how these 32-bit reductions are executed under the white-box encoding. Suppose two distinct lanes A and B are protected by the same linear matrix $\mathbf{M}$, yielding the encoded states $\tilde{A} = \mathbf{M} \cdot A$ and $\tilde{B} = \mathbf{M} \cdot B$. Because $\mathbf{M}$ functions is a pure linear mapping, it naturally distributes over the XOR operation as

$$\mathbf{M} \cdot (A \oplus B) = (\mathbf{M} \cdot A) \oplus (\mathbf{M} \cdot B).$$

B.3 Security analysis against structural ASA decomposition

The structural decomposition attacks [6–8, 11, 12], targeting Affine-Substitution-Affine (ASA) structures, is the main cryptanalysis method to evaluate the security of white-box implementations. These attacks rely heavily on the assumption that the **S**-layer is constructed by concatenating multiple small, independent S-boxes (e.g., 4-bit or 8-bit mappings). While our tensor-based implementation exhibits an ASA-like structure, where the **A** layers contain linear encodings $\mathbf{M}$ and the inner **S** layer consists of a non-linear logical-AND operation, it fundamentally breaks the prerequisites required for ASA decomposition.

In traditional ASA constructions, the non-linear layer operates on partitioned sub-blocks. In contrast, the MAndRX operation in Cube-f is constructed without the localized S-boxes. The bitwise logical-AND operations ($\wedge$), interleaved with cyclic shifts ($\gg$) and XORs ($\oplus$), create algebraic dependencies across the entire m -bit lane ($m = 32$).

When transformed to the tensor product space, the entire non-linear transition is linearized into a single, indivisible matrix multiplication as

$$\tilde{X}^{(i+1)} = \tilde{\mathbf{Q}}_i \cdot (\tilde{X}^{(i)} \otimes \tilde{X}^{(i)}).$$

The obfuscated round matrix $\tilde{\mathbf{Q}}_i \in \mathbb{F}_2^{m \times m^2}$ acts as a global quadratic mapping from a 1024-dimensional tensor space back to a 32-dimensional lane space. Because the non-linear terms are structurally obfuscated across the entire lane, there are no independent s -bit sub-blocks to isolate. Consequently, an attacker cannot partition the encoded matrix into smaller sub-functions to mount a divide-and-conquer structural decomposition.

To execute an ASA decomposition on this tensor-based scheme, an adversary must solve the linear equivalence problem. Specifically, given the public obfuscated matrix $\tilde{\mathbf{Q}}_i$, the attacker must recover the secret decoding tensor $(\mathbf{M}^{(i)} \otimes \mathbf{M}^{(i)})^{-1}$ and the encoding matrix $\mathbf{M}^{(i+1)}$ such that:

$$\tilde{\mathbf{Q}}_i = \mathbf{M}^{(i+1)} \cdot \mathbf{Q}_i \cdot (\mathbf{M}^{(i)} \otimes \mathbf{M}^{(i)})^{-1}$$

In an ASA decomposition, recovering the encodings requires solving equivalence relations within small algebraic groups, such as $\mathbb{F}_2^8$. The encoding matrix $\mathbf{M}^{(i)}$ in the tensor framework is randomly sampled from the full linear group $\mathbb{F}_2^{32}$, which contains approximately 2^{1024} elements. Furthermore, the attacker must solve this equivalence over the 32×1024 tensor expansion matrix. The problem of finding linear equivalences for global quadratic Boolean functions operating on 32-bit blocks (with no parallel S-box boundaries) is known to be computationally intractable. The absence of distinct algebraic boundaries, such as the discrete bytes in an AES state or isolated MixColumns operations, denies the adversary the structural properties required to construct overdefined systems of algebraic equations. Therefore, mapping the MAndRX operation to a unified tensor product space ensures that the white-box implementation can resist ASA structural decomposition.

B.4 Performance evaluation of the tensor-based implementation

In this section, we evaluate the efficiency of the tensor-based **Cube-f** implementation in terms of storage complexity and computational overhead. The evaluation is based on the design parameters of **Cube-f**, namely a state width of 1536 bits (6×8 lanes), a lane length of $m = 32$ bits, and the ZIP construction for the transformation, which evaluates a 9-round forward permutation and a 9-round inverse permutation, totaling 18 round evaluations per block.

Storage complexity. The primary storage requirement for our tensor-based implementation arises from the obfuscated round matrices $\tilde{\mathbf{Q}}_i \in \mathbb{F}_2^{m \times m^2}$. For each 32-bit lane, the implementation requires one obfuscated matrix per round. Let $N_L = 48$ be the total number of lanes. Because the ZIP construction computes the XOR sum of a 9-round forward evaluation and a 9-round backward evaluation, the total number of required round matrices per lane is $N_R = 9 + 9 = 18$. The storage complexity for the round matrices is calculated as

$$S_{matrices} = N_L \times N_R \times (m \cdot m^2) \text{ bits.}$$

Substituting $m = 32$, each matrix $\tilde{\mathbf{Q}}_i$ occupies $32 \times 1024 = 32,768$ bits (4 KB). For the entire 18-round ZIP execution across all 48 lanes, the total storage for round matrices is

$$48 \times 18 \times 4 \text{ KB} = 3456 \text{ KB} \approx 3.375 \text{ MB.}$$

Compared to the 107.7 MB required for the white-box KMAC implementation [17], our tensor-based construction reduces the storage footprint by over 96%.

Computational complexity. The execution time is dominated by the generation of the tensor product $X \otimes X$ and the subsequent matrix-vector multiplication for each round.

- **Tensor construction.** To compute $\tilde{X} \otimes \tilde{X}$ for a 32-bit lane, we generate all $m^2 = 1024$ quadratic combinations. Since $x_i \cdot x_j = x_j \cdot x_i$ and $x_i^2 = x_i$ over $\mathbb{F}_2$, an optimized implementation only needs to compute the unique $\frac{m(m+1)}{2} = 528$ terms. However, to maintain the strict $m \times m^2$ matrix structure for obfuscation, we assume the full 1024 terms are processed.
- **Matrix-vector multiplication.** Each round update involves a multiplication of a 32×1024 matrix by a 1024×1 vector. In $\mathbb{F}_2$, this corresponds to
 - $32 \times 1024 = 32,768$ bitwise AND operations.
 - $32 \times 1023 = 32,736$ bitwise XOR operations.

The total computational cost per state transformation **Cube-f** is

$$Cost_{\mathbf{F}} = N_L \times N_R \times (m \cdot m^2 \text{ ANDs} + m \cdot (m^2 - 1) \text{ XORs}) + Cost_{ZIP},$$

where $Cost_{ZIP}$ accounts for the final 1536-bit XOR addition between the forward and inverse branches. Substituting $N_L = 48$ and $N_R = 18$, the round evaluations require $864 \times 65,504 \approx 5.659 \times 10^7$ operations. Adding the 1536 XORs for the final ZIP merge, the implementation executes approximately 5.66×10^7 bitwise operations per 1536-bit block.